

Documento de Diseño Proyecto 1

- Juan Esteban Gomez
- Andres Gratz - 202514109
- Santiago Ismael Escobar Maidana- 202516956

Descripción de clases

Usuario

Representa a cualquier persona con acceso al sistema mediante credenciales.

Atributos:

- login: String
- password: String
- nombre: String
- juegosFavoritos: ArrayList<Juego>
- puntosFidelidad: int
- ventas: List<Venta>

Es una clase abstracta. No se instancian usuarios genéricos, solo tipos específicos como cliente, empleado o administrador.

Cliente

Representa al comensal que utiliza los servicios del café, realiza préstamos de juegos y consume productos.

Atributos:

- esNiño: boolean
- esJoven: boolean
- juegosReservados: int

Hereda de: Usuario

Responsabilidad: Reservar mesas, solicitar préstamos de juegos (máximo 2), comprar juegos o comida y acumular puntos de fidelidad.

Empleado

Representa al personal del café encargado de la operación diaria.

Atributos:

- codigoDescuento: String
- estáDeTurno: boolean
- turnos: List<Turno>

Hereda de: Usuario

Es una clase abstracta que se divide en roles operativos específicos.

Mesero

Tipo de empleado encargado de atender mesas, gestionar la caja y enseñar juegos difíciles.

Atributos:

- juegosConocidos: ArrayList<Juego>

Hereda de: Empleado

Responsabilidad: Atender pedidos, registrar ventas y brindar asesoría técnica en juegos etiquetados como "Difíciles".

Cocinero

Tipo de empleado encargado de la preparación de alimentos.

Hereda de: Empleado

Responsabilidad: Preparar los pedidos de pastelería y bebidas del menú.

Administrador

Usuario con facultades totales sobre la gestión financiera y de personal.

Hereda de: Usuario

Responsabilidad: Aprobar cambios de turno, gestionar inventarios (compras, reparaciones, bajas por robo) y generar informes de ventas segregados.

Turno

Representa la asignación horaria por día de los empleados, cada empleado puede tener muchos turnos a la semana.

Atributos:

- diaSemana: String

- estaAprobado: boolean

Responsabilidad: Organizar la jornada laboral y asegurar que se cumpla el personal mínimo requerido para operar.

Juego

Representa un título de mesa disponible en el catálogo del café.

Atributos:

- nombre: String
- anioPublicacion: int
- empresaMatriz: String
- minJugadores: int
- maxJugadores: int
- edadMinima: int
- categoria: String
- esDificil: boolean

copiaPrestamo

Representa una unidad física individual de un juego destinada exclusivamente al préstamo en sala.

Atributos:

- idUnico: String
- estado: String
- estaDisponible: Boolean

Responsabilidad: Mantener el registro individual del uso y el estado físico de cada copia del inventario de préstamo.

Mesa

Representa el espacio físico donde se ubican los clientes para consumir y jugar.

Atributos:

- idMesa: int
- capacidadMax: int
- ocupada: Boolean
- cantPersActuales: int
- hayNinos: boolean
- hayJovenes: boolean

Responsabilidad: Controlar la ocupación del local y validar que los juegos prestados sean aptos para los comensales presentes.

Prestamo

Representa la transacción temporal de uso de juegos de mesa por parte de un cliente o empleado.

Atributos:

- estado: String
- fechaHoraInicio: DateTime
- fechaHoraFin: DateTime

Responsabilidad: Registrar el historial de uso de los juegos, quién los tuvo y en qué tiempos.

Producto

Representa los elementos consumibles del menú de la cafetería.

Atributos:

- nombre: String
- precioBase: double

Es una clase abstracta para diferenciar entre tipos de productos.

Bebida

Tipo de producto líquido disponible en el menú.

Atributos:

- esAlcoholica: boolean
- esCaliente: boolean

Hereda de: Producto

Pasteleria

Tipo de producto sólido disponible en el menú.

Hereda de: Producto

Venta

Registro de una transacción comercial realizada en el establecimiento.

Atributos:

- idVenta: int
- fecha: Date
- subtotal: double
- impuestos: double
- propina: double
- total: double
- descuentoAplicado: int
- puntosGenerados: int

JuegoVenta

Representa el inventario de juegos destinados a la venta directa al público.

Atributos:

- precio: double
- stock: int

Responsabilidad: Gestionar la disponibilidad y el precio de los títulos comerciales, diferenciándolos del inventario de préstamo.

Relaciones del modelo

- **Relación entre Cliente y Mesa:** Un cliente debe ocupar una mesa para realizar préstamos de juegos. La mesa registra la cantidad de personas y la presencia de menores para validar la aptitud de los juegos y bebidas.
- **Relación entre Mesa y Juego:** Una mesa puede tener asociados simultáneamente hasta dos juegos del inventario de préstamos. Esta relación se bloquea si el juego no es apto para la edad de los comensales o el número de personas.
- **Relación entre Mesero y Juego:** Un mesero tiene una lista de juegos que conoce a cabalidad. Si un juego es "Difícil", se requiere idealmente un mesero con este conocimiento para realizar el préstamo.
- **Relación entre Empleado y Turno:** Cada empleado tiene asignado un turno semanal fijo. Los cambios o intercambios entre empleados requieren aprobación del administrador.
- **Relación entre Administrador e Inventarios:** El administrador gestiona dos inventarios separados: el de préstamo y el de venta. Puede mover juegos entre inventarios o darlos de baja por robo.
- **Relación entre Cliente y Prestamo:** Un cliente inicia un préstamo. El sistema debe registrar qué cliente es responsable de las copias mientras el préstamo esté activo.
- **Relación entre Prestamo y copiaPrestamo:** Un préstamo puede incluir de **1 a 2 copias** de juegos simultáneamente.
- **Relación entre Mesa y Prestamo:** Todo préstamo debe estar asociado a una mesa física para ser válido.

- **Relación entre Mesero y Prestamo:** Un mesero capacitado puede estar vinculado a un préstamo si el juego solicitado es "Difícil", para enseñar las reglas.

Reglas del dominio

1. Todo usuario debe poseer un login y un password para acceder al sistema.
2. Un cliente solo puede tener en préstamo un máximo de dos juegos simultáneamente.
3. El ingreso al café está limitado por una capacidad máxima; si se supera, la reserva de mesa es rechazada.
4. Los juegos de categoría "Acción" no pueden ser prestados a mesas que tengan bebidas calientes por motivos de seguridad.
5. Las bebidas alcohólicas no pueden ser despachadas a mesas donde se encuentren menores de 18 años.
6. Los juegos "Difíciles" pueden prestarse sin mesero capacitado solo bajo advertencia al cliente.
7. Las ventas de la tienda de juegos deben pagar un IVA del 19%.
8. Las ventas de cafetería deben pagar un impuesto al consumo del 8%.
9. Los empleados reciben un descuento del 20% en sus compras, y pueden compartir un 10% con otros clientes (no acumulable).
10. Para que el café opere legalmente, el turno debe contar con al menos 1 cocinero y 2 meseros simultáneamente.
11. Cada venta genera el 1% de su valor en puntos de fidelidad para el comprador, donde 1 punto equivale a 1 peso.
12. Los juegos que no se devuelven tras varios días deben marcarse como "Desaparecido" por el administrador.

Requerimientos Funcionales (RF)

Para Cliente

- **RF01 - Reservar Mesa:** El sistema debe permitir al cliente tomar una mesa indicando cuántas personas lo acompañan y si hay menores de edad.
- **RF02 - Consultar Catálogo:** El cliente debe poder visualizar los juegos disponibles para préstamo y venta.
- **RF03 - Solicitar Préstamo:** El cliente puede pedir hasta dos juegos simultáneamente si cumple con los requisitos de edad y capacidad de la mesa.
- **RF04 - Realizar Compra:** El cliente puede comprar juegos de la tienda o productos de cafetería.
- **RF05 - Gestionar Favoritos:** El cliente puede guardar una lista de sus juegos preferidos.

Para Empleado

- **RF06 - Consultar Turnos:** Los empleados pueden ver su horario semanal asignado.
- **RF07 - Solicitar Cambio de Turno:** Un empleado puede pedir un intercambio de turno con otro compañero o un cambio general.
- **RF08 - Sugerir Menú:** Cualquier empleado puede proponer nuevos platillos para el menú del café.
- **RF09 - Alquiler para Empleados:** Pueden alquilar juegos sin necesidad de mesa, siempre que no estén en su jornada laboral.
- **RF14 - Capacitación (Meseros):** Registrar y consultar los "Juegos Dificiles" en los que está capacitado para enseñar reglas.

Para Administrador

- **RF10 - Gestión de Inventario:** El administrador puede comprar, reabastecer, reparar o dar de baja juegos.
- **RF11 - Control de Personal:** El administrador debe aprobar o rechazar las solicitudes de cambio de turno.
- **RF12 - Generar Informes:** El sistema debe producir reportes de ventas detallados por rubro, fecha y desglose de impuestos/propinas.
- **RF13 - Actualizar Menú:** Potestad para agregar platillos o aceptar sugerencias de los empleados.

Restricciones del Proyecto

Restricciones de Dominio (Reglas de Negocio)

- **Capacidad del Local:** No se permiten más personas de las que soporta la capacidad máxima definida.
- **Seguridad y Edad:** No se despacha alcohol a mesas con menores de 18 años.
- **Seguridad de Juegos:** Está prohibido servir bebidas calientes en mesas que tengan juegos de la categoría "Acción".
- **Mínimo de Personal:** Cada turno debe contar con al menos 1 cocinero y 2 meseros.
- **Impuestos:** Los juegos de mesa graban un 19% de IVA, mientras que la cafetería graba un 8% de impuesto al consumo.

Restricciones Técnicas

- **Persistencia:** Toda la información debe almacenarse en archivos fuera de la carpeta del código fuente.
- **Tecnología:** Desarrollo en lenguaje Java.
- **Acceso:** Todo usuario requiere login y password para interactuar con el sistema.
- **Presupuesto:** El desarrollo del proyecto se puede ver limitado por el presupuesto que tiene la empresa destinado para el desarrollo. Si hay poco presupuesto es probable que se tengan que sacrificar funcionalidades y/o mantenibilidad.

Descripción de los Programas de Prueba

Se implementarán los siguientes programas de prueba para probar el programa.

Prueba de Carga: Un programa que cargue datos desde los archivos y demuestre que la información se mantiene entre ejecuciones.

Prueba de Ventas y Puntos: Un programa que realice una compra y muestre cómo se acumulan los puntos de fidelidad en el perfil del usuario.

Prueba de Gestión de Inventario: Un programa donde el administrador marque un juego como "robado" y se verifique que ya no aparece disponible para préstamo.

Diseño e implementación

Introducción

En esta segunda fase, **implementamos** la lógica de negocio necesaria para transformar el análisis inicial en una aplicación funcional. El sistema no solo gestiona transacciones, sino que garantiza el cumplimiento de reglas críticas como el control de aforo, la seguridad en el consumo de productos y la persistencia de datos. **Decidimos** priorizar la integridad de la información y la extensibilidad del código mediante una arquitectura orientada a objetos robusta.

2. Modelo de Diseño

2.1. Justificación de Decisiones Arquitectónicas

Tras analizar la retroalimentación de la Entrega 1, **realizamos cambios profundos** en la estructura:

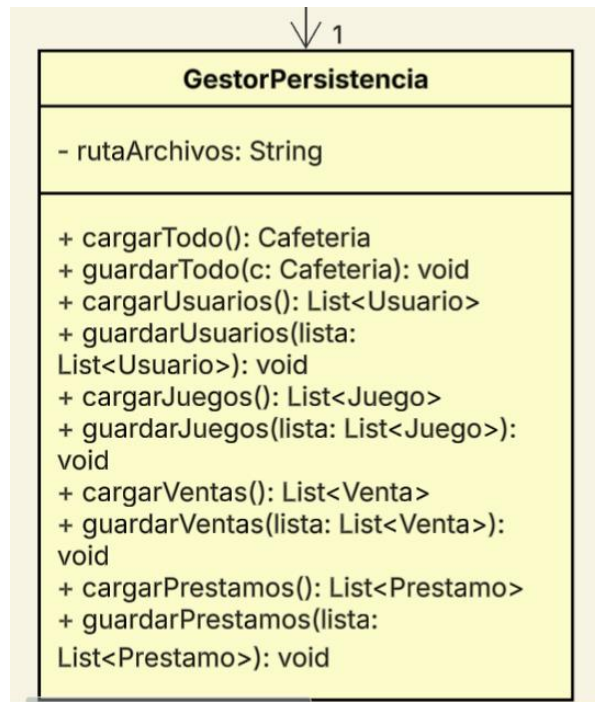
- **Clase Central Cafeteria:** Implementamos esta clase como el corazón del sistema. Esta clase paso a ser el centro de todo el cual redirecciona y de el parten las conexiones a todo el flujo del sistema. Decidimos que esta clase sea la encargada de gestionar el aforo máximo (capacidadMax) y la ocupación actual. Al centralizar esto, garantizamos que ningún cliente ingrese si el establecimiento está lleno, cumpliendo con la restricción de seguridad.
- **Refactorización de Juegos y Copias:** Aunque inicialmente el profesor sugirió unificar, **decidimos** mantener una distinción clara en el diseño de bajo nivel entre CopiaPrestamo y CopiaVenta. **Analizamos** que la lógica de un préstamo (circular,

con estados de "incompleto" o "desgastado") es intrínsecamente distinta a la de una venta a un cliente (salida definitiva del inventario). Sin embargo, ambas se asocian a la clase Juego para no duplicar la información técnica (autor, edad mínima, etc.). Sin embargo mantuvimos la posibilidad de que una copia de un juego que estaba en préstamo pase a ser una copia disponible para venta.

- **Jerarquía de Usuarios: Implementamos** una estructura de herencia donde Usuario es abstracta. Esto nos permitió aplicar polimorfismo en los procesos de login y asegurar que solo los Meseros puedan acceder a funciones de atención, mientras que el Administrador gestiona el inventario.
- **Control de Seguridad en Mesas: Decidimos** que la clase Mesa guarde el estado demográfico (hayNinos, hayJovenes). Esto permite que, antes de registrar un ItemVenta de tipo alcohol, el sistema verifique el estado de la mesa y bloquee la transacción si es necesario así como es solicitado en las reglas de negocio.
- **Consistencia de Atributos y Métodos:** Decidimos que cada mensaje en nuestros diagramas de secuencia (como actualizarPuntosFidelidad o estaDisponible) se convierta en un método public en Java, asegurando una trazabilidad del 100% entre el diseño y el código final.
- **Gestores:** Decidimos que existieran clases intermedias llamadas gestores, los cuales permiten que el flujo de la aplicación no sea descargado completamente sobre la clase cafetería, ya que esto violaría las reglas de diseño. Descargamos alguna lógica Importante sobre estos gestores, ya que permite tener mucho más desglosado el código y evitar que la clase cafetería quede supercargada, tener un flujo más limpio y tener los roles de diseño mucho más claros para cada clase.

3. Arquitectura de Persistencia

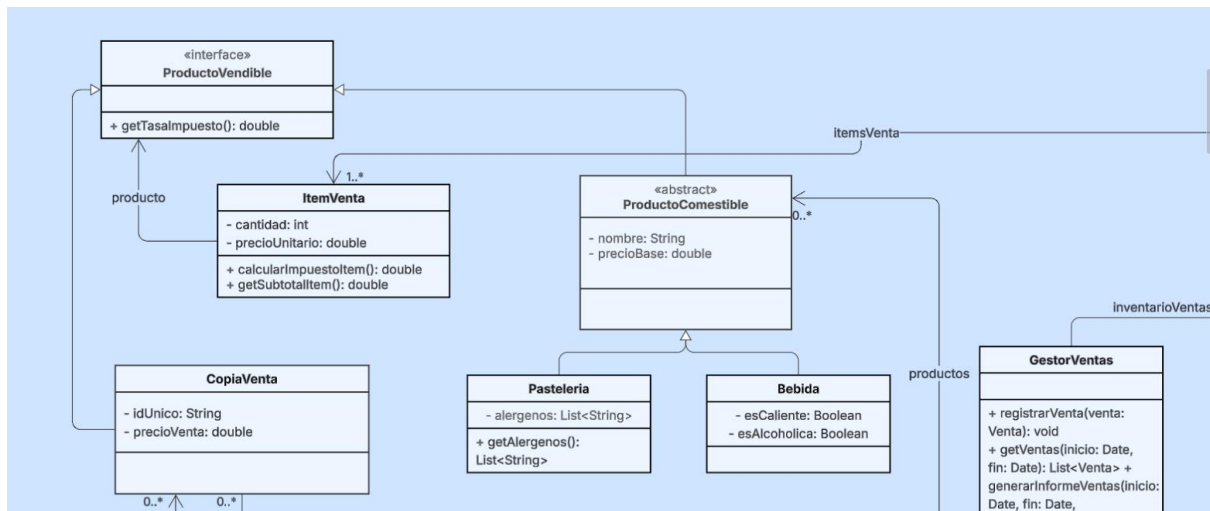
Implementamos un sistema de persistencia basado en archivos de texto plano (CSV), cumpliendo con la restricción de mantener los datos fuera de la carpeta src. Escogimos csv ya que existe bastante información en internet de como usar las librerías y implementar este tipo de persistencia, además permite editar fácilmente la información desde otro programa.



- **Estructura:** Creamos la clase **GestorPersistencia** encargada exclusivamente de la lectura y escritura.
- **Carga Inicial:** Al arrancar, `cargarTodo()` reconstruye el estado completo de la Cafeteria, incluyendo la lista de usuarios, el catálogo de juegos y el historial de ventas desde la ultima sesion.
- **Relaciones:** Para persistir relaciones (ej. qué juegos tiene reservados un cliente), **decidimos** guardar los IDs de los juegos en el archivo de usuarios y, en tiempo de ejecución, "re-vincular" los objetos.

4. Lógica de Impuestos y Finanzas

Para cumplir con los requerimientos funcionales del negocio, **diseñamos** la interfaz **ProductoVendible**:

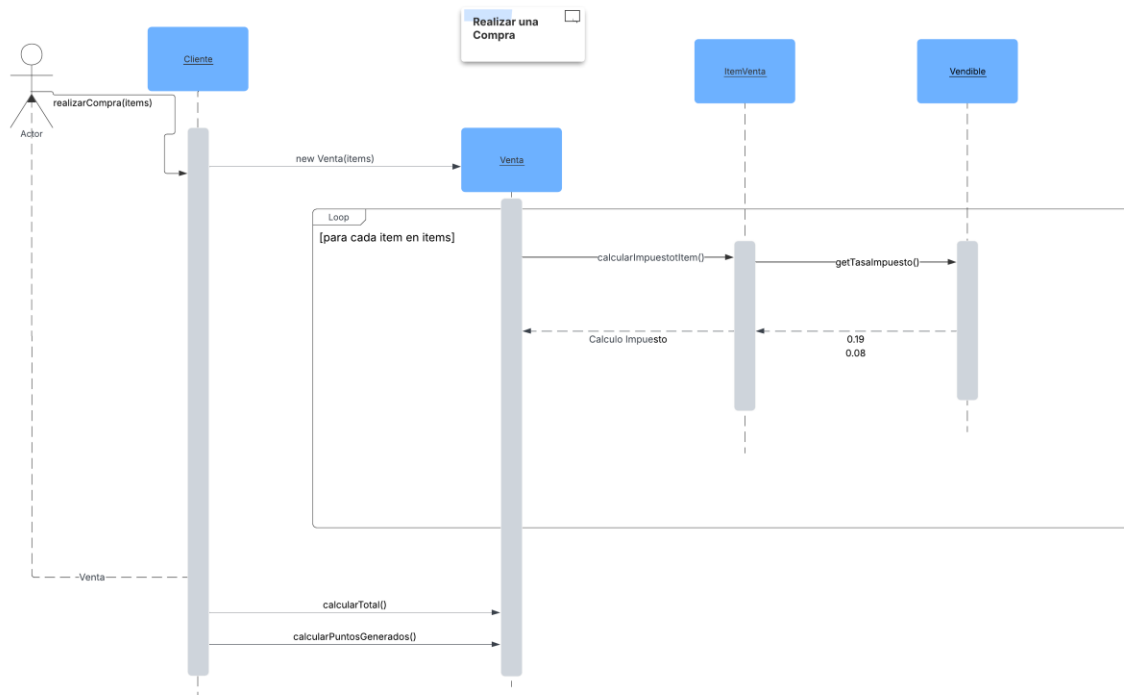


- **IVA (19%):** Aplicado automáticamente a través de la clase Juego y sus copias de venta.
- **Impuesto al Consumo (8%):** Aplicado a ProductoComestible (Bebidas y Pastelería).
- **Cálculo Automatizado:** El método calcularImpuestosTotales() en la clase Venta recorre los ItemVenta, identifica el tipo de producto y suma los gravámenes correspondientes, generando un desglose legal en la factura. Esto nos permitiera ser efectivos con los cálculos y el desglose de los impuestos/descuentos aplicados en una venta.

5. Diagramas de Secuencia (Validación de Lógica de Negocio)

Para asegurar que nuestro diseño de bajo nivel no solo fuera una estructura estática, sino un sistema capaz de soportar las reglas dinámicas del enunciado, **decidimos modelar** los siguientes tres flujos críticos:

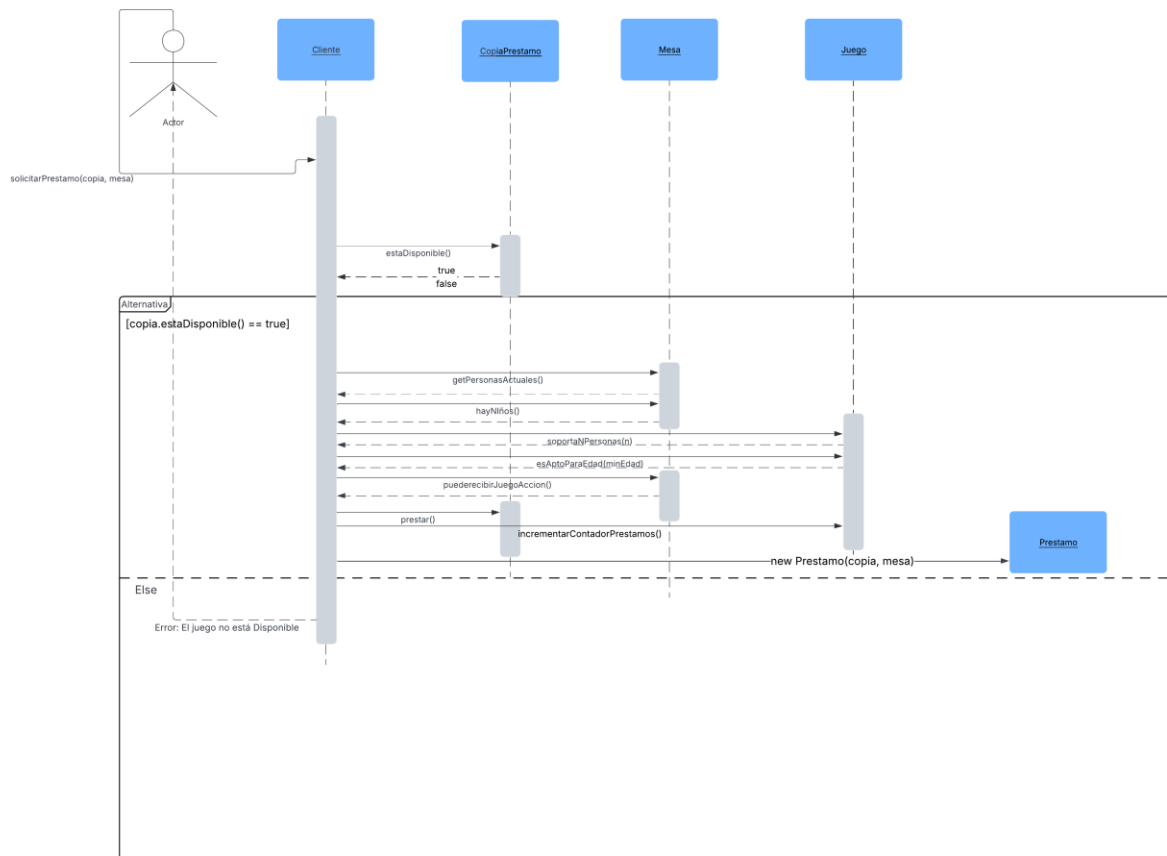
5.1. Realizar una Compra (Seguridad y Fidelización)



En este flujo, **implementamos** el rol del GestorVentas como el cerebro de la transacción.

- **Justificación de Diseño:** Analizamos que las restricciones de seguridad (como no vender alcohol a menores) no dependen solo de quién compra, sino del entorno (la mesa). Por ello, el diagrama muestra cómo el GestorVentas consulta a la Mesa los métodos `hayNinos()` y `hayJovenes()` antes de proceder.
- **Impacto en el Código:** Solo si estas validaciones pasan, se instancia el objeto `Venta`. Además, **decidimos** integrar aquí el sistema de puntos, permitiendo que el Cliente use sus beneficios (`usarPuntosFidelidad`) y acumule nuevos en la misma operación, garantizando la consistencia del estado financiero.

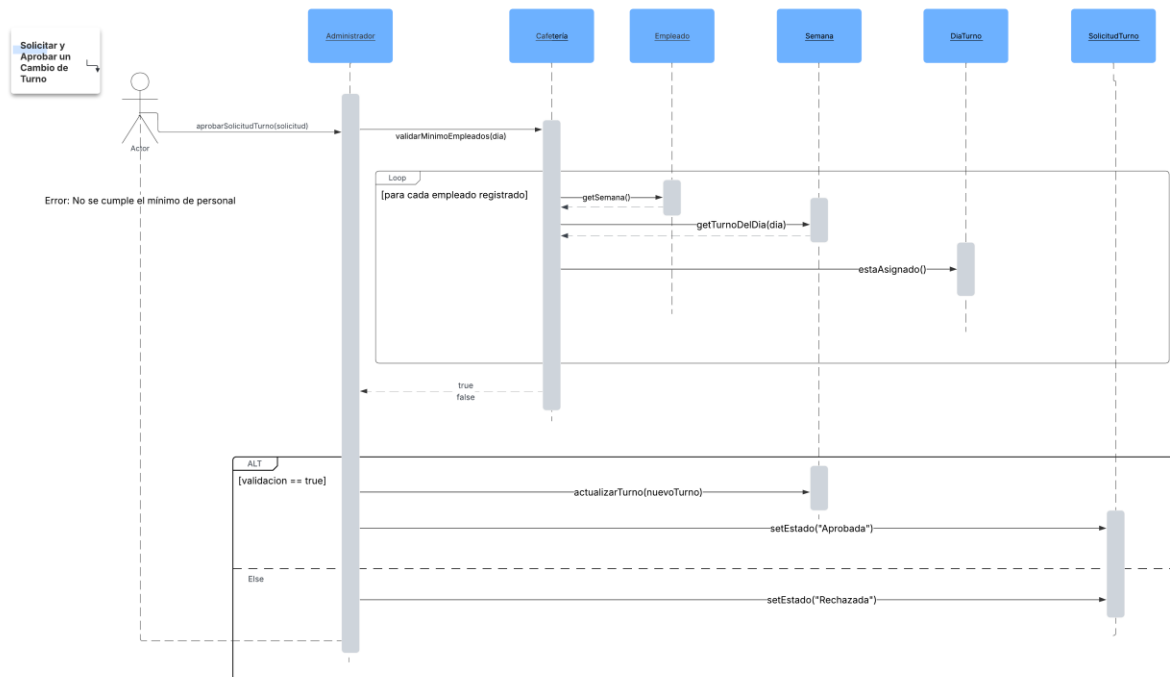
5.2. Préstamo de un Juego (Control de Inventario y Límites)



Este diagrama describe cómo gestionamos el recurso más valioso del café: los juegos de mesa.

- **Justificación de Diseño:** Para cumplir con la restricción de "máximo 2 juegos por cliente", decidimos que la Cafeteria sea la encargada de validar el estado del usuario antes de permitir el préstamo.
- **Lógica Implementada:** El flujo detalla una doble validación: primero, verificamos la capacidad del cliente (`getJuegosReservados() < 2`) y luego la disponibilidad física del juego (`copia.estaDisponible()`). Solo al cumplirse ambas, se crea el objeto `Prestamo` y se actualiza el contador del cliente, evitando fugas de inventario o abusos en el servicio.

5.3. Solicitar y Aprobar un Cambio de Turno (Gestión Operativa)



Este es un flujo avanzado que **decidimos incluir** para demostrar que nuestro sistema soporta la gestión de personal exigida (mínimo 1 cocinero y 2 meseros).

- **Justificación de Diseño:** Analizamos que los turnos no pueden ser modificados de forma aislada por los empleados. Por eso, introducimos la clase Semana como el validador de horarios.
- **Flujo de Trabajo:** Un Empleado inicia la solicitud, pero el intercambio solo se concreta si el otro empleado acepta y si la Semana ejecuta el método `actualizarTurno()`. Esto nos permite, en la implementación, incluir una lógica que verifique que el cambio no deje al café sin el personal mínimo requerido para operar.

6. Documentación de Programas de Prueba

Analizamos los errores pasados y **decidimos** formalizar las pruebas con el siguiente rigor:

Prueba 1: Persistencia y Consistencia de Datos

- **Caso:** Cerrar la aplicación tras crear un nuevo usuario y volver a abrirla.
- **Entrada:** Registro de usuario "Santi123", contraseña "clave1", tipo "Cliente".
- **Salida Esperada:** El archivo usuarios.txt debe contener la nueva línea. Al reiniciar, el sistema debe permitir el login de "Santi123".

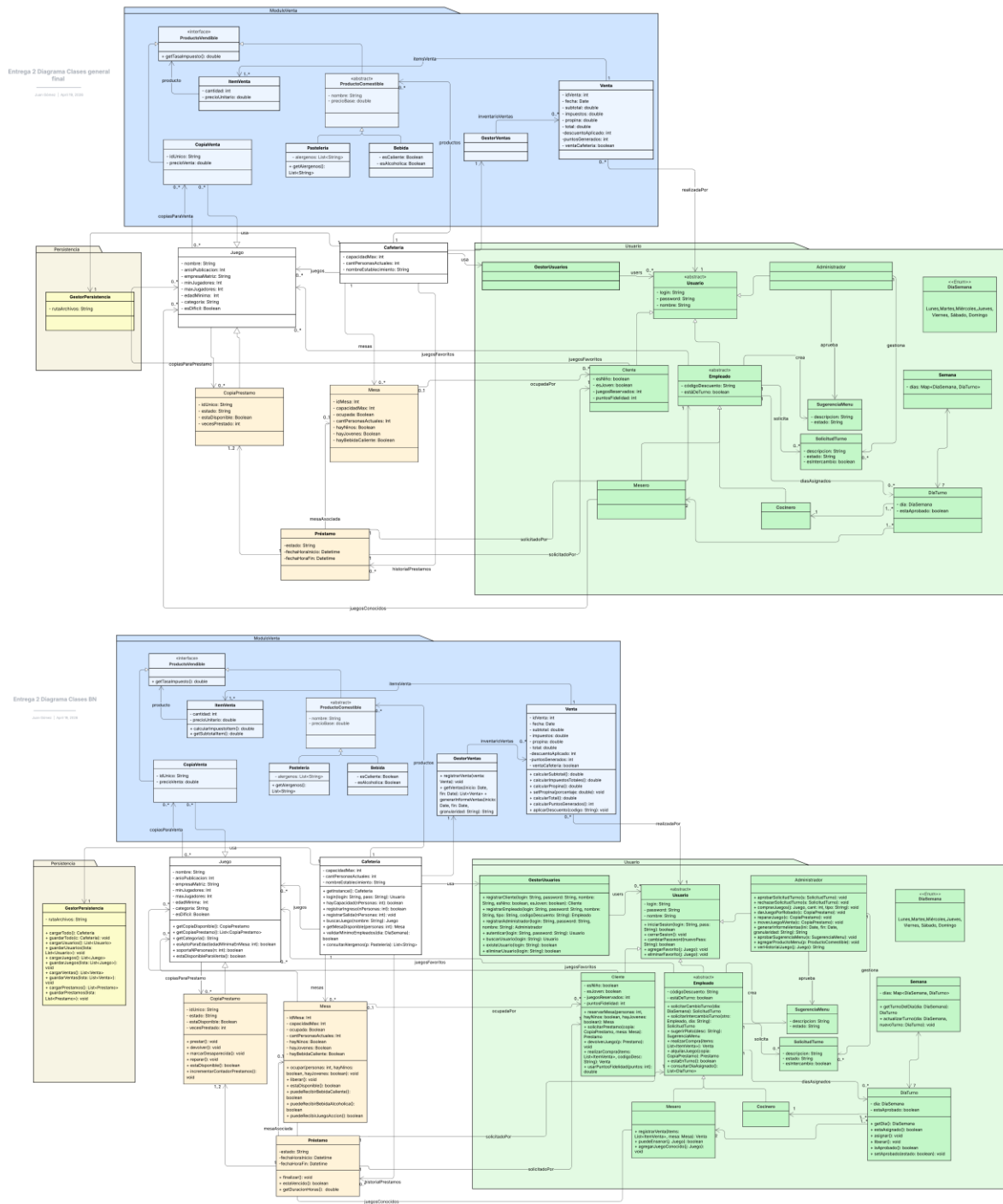
Prueba 2: Restricción de Seguridad (Bebidas Calientes)

- **Caso:** Intentar pedir un café caliente en una mesa con un juego de "Acción".
- **Entrada:** Mesa 5 con Juego "Ritmo y Bola" (Categoría: Acción). Pedido: "Capuchino" (esCaliente: true).
- **Salida Esperada:** Mensaje de error: "Protocolo de seguridad: No se permiten bebidas calientes cerca de juegos de acción para evitar daños al material".

Prueba 3: Gestión de Aforo

- **Caso:** Ingreso de grupo que supera la capacidad.
- **Entrada:** capacidadMax: 20. cantPersonasActuales: 18. Intento de ingreso de 4 personas.
- **Salida Esperada:** Bloqueo de ingreso. Mensaje: "Capacidad insuficiente. Solo quedan 2 cupos disponibles".

7. Diagramas de clase (alto nivel y normal)



Los archivos estan en el git.

8. Manual de Ejecución

Importar el proyecto en Eclipse.

Asegurarse de que la carpeta data/ esté en la raíz del proyecto (fuera de src).

Ejecutar Main.java.

Credenciales de prueba: Admin / admin123.